사용 예제 및 실습 가이드

개요

이 문서는 arkworks-rs/poly-commit 라이브러리를 실제로 사용하는 방법을 단계별 예제와 함께 설명합니다. 기본적인 setup부터 commit, open, verify까지의 전체 과정을 다루며, 고급 기능인 batch operations, hiding commitments, degree bounds 등의 사용법도 포함합니다.

기본 사용법

1. 의존성 설정

먼저 `Cargo.toml` 에 필요한 의존성을 추가합니다:

```
[dependencies]

ark-poly-commit = "0.5.0"

ark-bls12-381 = { version = "0.5.0", features = ["curve"] }

ark-poly = "0.5.0"

ark-ff = "0.5.0"

ark-std = "0.5.0"

ark-crypto-primitives = { version = "0.5.0", features = ["sponge"] }

rand = "0.8"
```

2. 기본 import 및 타입 정의

```
use  ark_poly_commit::{
marlin_pc::MarlinKZG10,
PolynomialCommitment,
LabeledPolynomial,
LabeledCommitment,
QuerySet,
Evaluations
};
use  ark_bls12_381::{Bls12_381, Fr};
use  ark_poly::{
univariate::DensePolynomial,
DenseUVPolynomial,
Polynomial
};
use ark_ff::{Field, UniformRand};
use ark_std::{test_rng, collections::BTreeMap};
use ark_crypto_primitives::sponge::poseidon::PoseidonSponge;


// 타입 별칭 정의
type UniPoly = DensePolynomial<Fr>;
type PC = MarlinKZG10<Bls12_381, UniPoly>;
type Commitment = <PC as PolynomialCommitment<Fr, UniPoly>>::Commitment;
```

```
type Proof = <PC as PolynomialCommitment<Fr, UniPoly>>::Proof;
```

3. 기본 워크플로우: Setup → Commit → Open → Verify

3.1 Setup 단계

```
fn basic_setup_example() -> Result<(), Box<dyn std::error::Error>> {

let rng = &mut test_rng();

// 1. Universal Parameters 생성

// 최대 차수 100인 다항식을 지원하는 파라미터 생성

let max_degree = 100;

let pp = PC::setup(max_degree, None, rng)?;

println!("Universal parameters 생성 완료. 최대 차수: {}", pp.max_degree());

// 2. Committer Key와 Verifier Key 생성

let supported_degree = 50; // 실제 사용할 최대 차수

let supported_hiding_bound = 10; // hiding을 위한 쿼리 수

let enforced_degree_bounds = None; // degree bound 강제 없음

let (ck, vk) = PC::trim(

&pp,

supported_degree,

supported_hiding_bound,

enforced_degree_bounds

)?;

println!("Committer key와 Verifier key 생성 완료");
```

```
    println!("지원 차수: {}", ck.supported_degree());

    Ok(())

    }
```

3.2 다항식 생성 및 Commit 단계

```
    fn basic_commit_example() -> Result<(), Box<dyn std::error::Error>> {

    let rng = &mut test_rng();

    // Setup

    let max_degree = 100;

    let pp = PC::setup(max_degree, None, rng)?;

    let (ck, vk) = PC::trim(&pp, 50, 10, None)?;

    // 1. 다항식 생성

    // f(x) = 3x^3 + 2x^2 + x + 5

    let coeffs = vec![Fr::from(5u64), Fr::from(1u64), Fr::from(2u64), Fr::from(3u64)

    let polynomial = DensePolynomial::from_coefficients_vec(coeffs);

    // 2. LabeledPolynomial로 래핑

    let labeled_poly = LabeledPolynomial::new(

    "my_polynomial".to_string(),

    polynomial,

    None, // degree bound 없음

    Some(5) // 최대 5번의 쿼리에 대해 hiding 제공

    );

    // 3. Commit 수행
```

```
let (commitments, commitment_states) = PC::commit(
&ck,
&[labeled_poly.clone()],
Some(rng)
)?;
println!("다항식 커밋 완료");
println!("커밋 개수: {}", commitments.len());
Ok(())
}
```

3.3 Open 및 Verify 단계

```
fn basic_open_verify_example() -> Result<(), Box<dyn std::error::Error>> {
let rng = &mut test_rng();
// Setup 및 Commit (이전 단계)
let max_degree = 100;
let pp = PC::setup(max_degree, None, rng)?;
let (ck, vk) = PC::trim(&pp, 50, 10, None)?;
let coeffs = vec![Fr::from(5u64), Fr::from(1u64), Fr::from(2u64), Fr::from(3u64)
let polynomial = DensePolynomial::from_coefficients_vec(coeffs);
let labeled_poly = LabeledPolynomial::new(
"my_polynomial".to_string(),
polynomial.clone(),
```

```
None,

Some(5)

);

let (commitments, commitment_states) = PC::commit(&ck, &[labeled_poly.clone()],

// 1. 평가할 점 선택

let evaluation_point = Fr::from(10u64); // x = 10에서 평가

// 2. 실제 다항식 값 계산 (검증용)

let expected_value = polynomial.evaluate(&evaluation_point);

println!("x = 10에서의 실제 값: {:?}", expected_value);

// 3. Sponge 초기화 (Fiat-Shamir를 위한 랜덤 오라클)

let mut sponge = create_sponge();

// 4. Opening proof 생성

let proof = PC::open(

&ck,

&[labeled_poly],

&commitments,

&evaluation_point,

&mut sponge.clone(),

&commitment_states,

Some(rng)

)?;

println!("Opening proof 생성 완료");

// 5. Proof 검증
```

```
let is_valid = PC::check(
&vk,
&commitments,
&evaluation_point,
&[expected_value],
&proof,
&mut sponge,
Some(rng)
)?;
println!("검증 결과: {}", if is_valid { "성공" } else { "실패" });
assert!(is_valid);
Ok(())
}


// Poseidon sponge 생성 헬퍼 함수
fn create_sponge() -> PoseidonSponge<Fr> {
use ark_crypto_primitives::sponge::poseidon::PoseidonConfig;
// 테스트용 파라미터 (실제 운영에서는 안전한 파라미터 사용 필요)
let full_rounds = 8;
let partial_rounds = 31;
let alpha = 17;
let mds = vec![
vec![Fr::from(1u64), Fr::from(0u64), Fr::from(1u64)],
```

```
vec![Fr::from(1u64), Fr::from(1u64), Fr::from(0u64)],

vec![Fr::from(0u64), Fr::from(1u64), Fr::from(1u64)],

];

let mut ark = Vec::new();

let mut rng = test_rng();

for _ in 0..(full_rounds + partial_rounds) {

let mut round_constants = Vec::new();

for _ in 0..3 {

round_constants.push(Fr::rand(&mut rng));

}

ark.push(round_constants);

}

let config = PoseidonConfig::new(full_rounds, partial_rounds, alpha, mds, ark, 2

PoseidonSponge::new(&config)

}
```

고급 기능 사용법

1. Batch Operations

여러 다항식을 동시에 처리하여 효율성을 높이는 방법:

```
fn batch_operations_example() -> Result<(), Box<dyn std::error::Error>> {

let rng = &mut test_rng();
```

```
// Setup

let max_degree = 100;

let pp = PC::setup(max_degree, None, rng)?;

let (ck, vk) = PC::trim(&pp, 50, 10, None)?;

// 1. 여러 다항식 생성

let mut polynomials = Vec::new();

for i in 0..3 {

let coeffs: Vec<Fr> = (0..=i+2).map(|j| Fr::from((i*10 + j) as u64)).collect();

let poly = DensePolynomial::from_coefficients_vec(coeffs);

let labeled_poly = LabeledPolynomial::new(

format!("poly_{}", i),

poly,

None,

Some(5)

);

polynomials.push(labeled_poly);

}

// 2. Batch commit

let (commitments, commitment_states) = PC::commit(&ck, &polynomials, Some(rng))?

println!("Batch commit 완료: {} 개의 다항식", polynomials.len());

// 3. Query set 구성 (여러 점에서 여러 다항식 평가)

let mut query_set = QuerySet::new();

let mut expected_evaluations = Evaluations::new();
```

```
let points = vec![Fr::from(5u64), Fr::from(10u64)];

for (point_idx, point) in points.iter().enumerate() {

for (poly_idx, poly) in polynomials.iter().enumerate() {

let point_label = format!("point_{}", point_idx);

let poly_label = poly.label().clone();

// Query set에 추가

query_set.insert((poly_label.clone(), (point_label.clone(), *point)));

// 예상 값 계산

let value = poly.evaluate(point);

expected_evaluations.insert((poly_label, *point), value);

}

}

// 4. Batch open

let mut sponge = create_sponge();

let batch_proof = PC::batch_open(

&ck,

&polynomials,

&commitments,

&query_set,

&mut sponge.clone(),

&commitment_states,

Some(rng)

)?;
```

```
    println!("Batch opening proof 생성 완료");

    // 5. Batch verify

    let is_valid = PC::batch_check(

    &vk,

    &commitments,

    &query_set,

    &expected_evaluations,

    &batch_proof,

    &mut sponge,

    rng

    )?;

    println!("Batch 검증 결과: {}", if is_valid { "성공" } else { "실패" });

    assert!(is_valid);

    Ok(())

    }
```

2. Hiding Commitments

커밋의 내용을 숨기는 hiding 기능 사용법:

```
    fn hiding_commitments_example() -> Result<(), Box<dyn std::error::Error>> {

    let rng = &mut test_rng();

    // Setup

    let max_degree = 100;
```

```
let pp = PC::setup(max_degree, None, rng)?;

let (ck, vk) = PC::trim(&pp, 50, 20, None)?; // hiding_bound를 크게 설정

// 1. Hiding polynomial 생성

let coeffs = vec![Fr::from(1u64), Fr::from(2u64), Fr::from(3u64)];

let polynomial = DensePolynomial::from_coefficients_vec(coeffs);

let hiding_poly = LabeledPolynomial::new(

"hiding_polynomial".to_string(),

polynomial.clone(),

None,

Some(15) // 15번의 쿼리까지 hiding 제공

);

// 2. Non-hiding polynomial (비교용)

let non_hiding_poly = LabeledPolynomial::new(

"non_hiding_polynomial".to_string(),

polynomial.clone(),

None,

None // hiding 없음

);

// 3. 각각 커밋

let (hiding_commitments, hiding_states) = PC::commit(

&ck,

&[hiding_poly.clone()],

Some(rng)
```

```
)?;

let (non_hiding_commitments, non_hiding_states) = PC::commit(

&ck,

&[non_hiding_poly.clone()],

Some(rng)

)?;

println!("Hiding commitment: {:?}", hiding_commitments[0]);

println!("Non-hiding commitment: {:?}", non_hiding_commitments[0]);

// 4. 같은 점에서 평가 및 검증

let point = Fr::from(7u64);

let expected_value = polynomial.evaluate(&point);

let mut sponge1 = create_sponge();

let mut sponge2 = create_sponge();

// Hiding proof

let hiding_proof = PC::open(

&ck,

&[hiding_poly],

&hiding_commitments,

&point,

&mut sponge1.clone(),

&hiding_states,

Some(rng)

)?;
```

```
// Non-hiding proof

let non_hiding_proof = PC::open(

&ck,

&[non_hiding_poly],

&non_hiding_commitments,

&point,

&mut sponge2.clone(),

&non_hiding_states,

Some(rng)

)?;

// 5. 검증

let hiding_valid = PC::check(

&vk,

&hiding_commitments,

&point,

&[expected_value],

&hiding_proof,

&mut sponge1,

Some(rng)

)?;

let non_hiding_valid = PC::check(

&vk,

&non_hiding_commitments,
```

```
&point,
&[expected_value],
&non_hiding_proof,
&mut sponge2,
Some(rng)
)?;
println!("Hiding commitment 검증: {}", hiding_valid);
println!("Non-hiding commitment 검증: {}", non_hiding_valid);
assert!(hiding_valid && non_hiding_valid);
Ok(())
}
```

3. Degree Bounds

다항식의 차수 제한을 강제하는 기능:

```
fn degree_bounds_example() -> Result<(), Box<dyn std::error::Error>> {
let rng = &mut test_rng();
// Setup
let max_degree = 100;
let pp = PC::setup(max_degree, None, rng)?;
// Degree bounds 설정
let degree_bounds = vec![10, 20, 30]; // 각각 10, 20, 30차 제한
let (ck, vk) = PC::trim(&pp, 50, 10, Some(&degree_bounds))?;
```

```
// 1. Degree bound가 있는 다항식들 생성

let mut polynomials = Vec::new();

// 10차 제한 다항식 (실제로는 8차)

let coeffs1: Vec<Fr> = (0..9).map(|i| Fr::from(i as u64)).collect();

let poly1 = DensePolynomial::from_coefficients_vec(coeffs1);

let labeled_poly1 = LabeledPolynomial::new(

"bounded_poly_10".to_string(),

poly1,

Some(10), // 10차 제한

Some(5)

);

polynomials.push(labeled_poly1);

// 20차 제한 다항식 (실제로는 15차)

let coeffs2: Vec<Fr> = (0..16).map(|i| Fr::from((i * 2) as u64)).collect();

let poly2 = DensePolynomial::from_coefficients_vec(coeffs2);

let labeled_poly2 = LabeledPolynomial::new(

"bounded_poly_20".to_string(),

poly2,

Some(20), // 20차 제한

Some(5)

);

polynomials.push(labeled_poly2);

// 2. Commit
```

```
let (commitments, commitment_states) = PC::commit(&ck, &polynomials, Some(rng))?

println!("Degree bound가 적용된 {} 개의 다항식 커밋 완료", polynomials.len());

// 3. 검증을 위한 쿼리

let point = Fr::from(5u64);

let mut query_set = QuerySet::new();

let mut expected_evaluations = Evaluations::new();

for (i, poly) in polynomials.iter().enumerate() {

let point_label = format!("eval_{}", i);

query_set.insert((poly.label().clone(), (point_label, point)));

let value = poly.evaluate(&point);

expected_evaluations.insert((poly.label().clone(), point), value);

}

// 4. Batch open 및 verify

let mut sponge = create_sponge();

let batch_proof = PC::batch_open(

&ck,

&polynomials,

&commitments,

&query_set,

&mut sponge.clone(),

&commitment_states,

Some(rng)

)?;
```

```
let is_valid = PC::batch_check(
&vk,
&commitments,
&query_set,
&expected_evaluations,
&batch_proof,
&mut sponge,
rng
)?;
println!("Degree bound 검증 결과: {}", if is_valid { "성공" } else { "실패" });
assert!(is_valid);
Ok(())
}
```

스킴별 성능 특성 및 선택 기준

1. 스킴 비교표

스킴	Setup 타입	Commitment 크기	Proof 크기	Verifier 시간	주요 특징
KZG10	Trusted	O(1)	O(1)	O(1)	가장 효율적, pairing 필요
Marlin PC	Trusted	O(1)	O(1)	O(1)	Degree bound 강제, hiding 지원
Sonic PC	Trusted	O(1)	O(log n)	O(log n)	Batch verification 최적화
IPA	Transparent	O(1)	O(log n)	O(n)	Pairing 불필요, 느린 검증

| **Hyrax** | Transparent | O(√n) | O(√n) | O(√n) | Multilinear polynomial |

| **Linear Codes** | Transparent | O(√n) | O(√n) | O(√n) | Post-quantum 안전성 |

2. 스킴 선택 가이드

KZG10을 선택해야 하는 경우:

```
// 최고 성능이 필요하고 trusted setup을 수용할 수 있는 경우
use ark_poly_commit::kzg10::KZG10;

fn when_to_use_kzg10() {
println!("KZG10 사용 권장 상황:");
println!("- 최소한의 proof 크기가 필요한 경우");
println!("- 빠른 verification이 중요한 경우");
println!("- Trusted setup을 수용할 수 있는 경우");
println!("- Pairing-friendly curve를 사용할 수 있는 경우");
}
```

Marlin PC를 선택해야 하는 경우:

```
// Degree bound 강제와 hiding이 필요한 경우
use ark_poly_commit::marlin_pc::MarlinKZG10;

fn when_to_use_marlin_pc() {
println!("Marlin PC 사용 권장 상황:");
```

```
    println!("- Degree bound 강제가 필요한 경우");

    println!("- Hiding commitment가 필요한 경우");

    println!("- Zero-knowledge proof 시스템에서 사용하는 경우");

    println!("- KZG10의 모든 장점 + 추가 기능이 필요한 경우");

    }
```

IPA를 선택해야 하는 경우:

```
    // Transparent setup이 필요한 경우

    use ark_poly_commit::ipa_pc::InnerProductArgPC;


    fn when_to_use_ipa() {

    println!("IPA 사용 권장 상황:");

    println!("- Trusted setup을 피하고 싶은 경우");

    println!("- Pairing을 지원하지 않는 curve를 사용하는 경우");

    println!("- Verification 시간이 중요하지 않은 경우");

    println!("- Transparency가 가장 중요한 요구사항인 경우");

    }
```

3. 성능 벤치마크 예제

```
    use std::time::Instant;


    fn performance_comparison_example() -> Result<(), Box<dyn std::error::Error>> {
```

```
let rng = &mut test_rng();

let max_degree = 1000;

// 테스트용 다항식 생성

let coeffs: Vec<Fr> = (0..=max_degree).map(|i| Fr::from(i as u64)).collect();

let polynomial = DensePolynomial::from_coefficients_vec(coeffs);

println!("성능 비교 테스트 (차수: {})", max_degree);

println!("=" * 50);

// KZG10 성능 측정

{

let start = Instant::now();

let pp = PC::setup(max_degree, None, rng)?;

let setup_time = start.elapsed();

let start = Instant::now();

let (ck, vk) = PC::trim(&pp, max_degree, 10, None)?;

let trim_time = start.elapsed();

let labeled_poly = LabeledPolynomial::new(

"test_poly".to_string(),

polynomial.clone(),

None,

Some(5)

);

let start = Instant::now();

let (commitments, states) = PC::commit(&ck, &[labeled_poly.clone()], Some(rng))?
```

```
let commit_time = start.elapsed();

let point = Fr::from(42u64);

let mut sponge = create_sponge();

let start = Instant::now();

let proof = PC::open(

&ck,

&[labeled_poly],

&commitments,

&point,

&mut sponge.clone(),

&states,

Some(rng)

)?;

let open_time = start.elapsed();

let expected_value = polynomial.evaluate(&point);

let start = Instant::now();

let is_valid = PC::check(

&vk,

&commitments,

&point,

&[expected_value],

&proof,

&mut sponge,
```

```
Some(rng)
)?;
let verify_time = start.elapsed();
println!("Marlin PC (KZG10 기반) 성능:");
println!(" Setup: {:?}", setup_time);
println!(" Trim: {:?}", trim_time);
println!(" Commit: {:?}", commit_time);
println!(" Open: {:?}", open_time);
println!(" Verify: {:?}", verify_time);
println!(" 검증 결과: {}", is_valid);
println!();
}
Ok(())
}
```

일반적인 실수와 해결책

1. Degree Bound 관련 실수

```
fn common_degree_bound_mistakes() {
println!("일반적인 Degree Bound 실수들:");
// 실수 1: degree bound가 실제 차수보다 작음
println!("❌ 실수: degree_bound < 실제 다항식 차수");
```

```
println!("해결책: degree_bound는 실제 차수보다 크거나 같아야 함");

// 실수 2: trim 시 degree bound를 지정하지 않음

println!("❌ 실수: degree bound가 필요한 다항식인데 trim 시 None 전달");

println!("해결책: enforced_degree_bounds 파라미터에 적절한 값 전달");

// 실수 3: hiding bound 설정 오류

println!("❌ 실수: hiding_bound > 쿼리 수");

println!("해결책: hiding_bound는 예상 쿼리 수보다 작거나 같게 설정");

}
```

2. Sponge 사용 관련 실수

```
fn common_sponge_mistakes() {

println!("일반적인 Sponge 사용 실수들:");

println!("❌ 실수: 같은 sponge 인스턴스를 open과 check에서 재사용");

println!("해결책: 각각 새로운 sponge 인스턴스 사용 또는 clone 사용");

println!("❌ 실수: 안전하지 않은 Poseidon 파라미터 사용");

println!("해결책: 운영 환경에서는 검증된 파라미터 사용");

}
```

3. 메모리 및 성능 최적화

```
fn optimization_tips() {

println!("성능 최적화 팁:");
```

```
println!("✅ Batch operations 사용으로 amortized cost 활용");
println!("✅ 적절한 degree bound 설정으로 불필요한 계산 방지");
println!("✅ Hiding bound를 실제 필요한 만큼만 설정");
println!("✅ Universal parameters 재사용으로 setup cost 절약");
println!("✅ 병렬 처리가 가능한 경우 parallel feature 활용");
}
```

실제 사용 시나리오

1. Zero-Knowledge Proof 시스템에서의 활용

```
fn zk_proof_scenario() -> Result<(), Box<dyn std::error::Error>> {
let rng = &mut test_rng();
// ZK proof에서 일반적인 시나리오:
// 1. Prover가 witness polynomial을 커밋
// 2. Verifier가 random challenge 제공
// 3. Prover가 challenge point에서 opening proof 제공
let max_degree = 100;
let pp = PC::setup(max_degree, None, rng)?;
let (ck, vk) = PC::trim(&pp, max_degree, 20, None)?;
// Witness polynomial (비밀 정보)
let witness_coeffs: Vec<Fr> = (0..50).map(|_| Fr::rand(rng)).collect();
let witness_poly = DensePolynomial::from_coefficients_vec(witness_coeffs);
```

```
let labeled_witness = LabeledPolynomial::new(
"witness".to_string(),
witness_poly.clone(),
None,
Some(10) // hiding 적용
);
// 1. Prover: witness 커밋
let (commitments, states) = PC::commit(&ck, &[labeled_witness.clone()], Some(rng
println!("Prover: witness polynomial 커밋 완료");
// 2. Verifier: random challenge 생성
let challenge = Fr::rand(rng);
println!("Verifier: challenge point 생성: {:?}", challenge);
// 3. Prover: opening proof 생성
let mut sponge = create_sponge();
let proof = PC::open(
&ck,
&[labeled_witness],
&commitments,
&challenge,
&mut sponge.clone(),
&states,
Some(rng)
)?;
```

```
let witness_value = witness_poly.evaluate(&challenge);

println!("Prover: opening proof 생성 완료");

// 4. Verifier: proof 검증

let is_valid = PC::check(

&vk,

&commitments,

&challenge,

&[witness_value],

&proof,

&mut sponge,

Some(rng)

)?;

println!("Verifier: 검증 결과 = {}", is_valid);

assert!(is_valid);

Ok(())

}
```

2. 다중 당사자 계산에서의 활용

```
fn multi_party_computation_scenario() -> Result<(), Box<dyn std::error::Error>>

let rng = &mut test_rng();

// MPC 시나리오: 여러 당사자가 각자의 입력을 커밋하고

// 나중에 특정 점에서 공개하여 결과를 계산
```

```
let max_degree = 50;

let pp = PC::setup(max_degree, None, rng)?;

let (ck, vk) = PC::trim(&pp, max_degree, 15, None)?;

// 3명의 참가자가 각자의 다항식을 커밋

let mut all_polynomials = Vec::new();

let mut all_commitments = Vec::new();

let mut all_states = Vec::new();

for party_id in 0..3 {

// 각 당사자의 비밀 다항식

let coeffs: Vec<Fr> = (0..10).map(|i| Fr::from((party_id * 10 + i) as u64)).coll

let party_poly = DensePolynomial::from_coefficients_vec(coeffs);

let labeled_poly = LabeledPolynomial::new(

format!("party_{}_input", party_id),

party_poly,

None,

Some(5)

);

// 개별 커밋

let (commitments, states) = PC::commit(&ck, &[labeled_poly.clone()], Some(rng))?

all_polynomials.push(labeled_poly);

all_commitments.extend(commitments);

all_states.extend(states);

println!("Party {}: 입력 커밋 완료", party_id);
```

```
}

// 공통 평가 점에서 모든 값을 공개

let evaluation_point = Fr::from(5u64);

let mut query_set = QuerySet::new();

let mut expected_evaluations = Evaluations::new();

for (i, poly) in all_polynomials.iter().enumerate() {

let point_label = format!("reveal_{}", i);

query_set.insert((poly.label().clone(), (point_label, evaluation_point)));

let value = poly.evaluate(&evaluation_point);

expected_evaluations.insert((poly.label().clone(), evaluation_point), value);

}

// Batch opening

let mut sponge = create_sponge();

let batch_proof = PC::batch_open(

&ck,

&all_polynomials,

&all_commitments,

&query_set,

&mut sponge.clone(),

&all_states,

Some(rng)

)?;

// 검증
```

```
let is_valid = PC::batch_check(
&vk,
&all_commitments,
&query_set,
&expected_evaluations,
&batch_proof,
&mut sponge,
rng
)?;
println!("MPC 결과 검증: {}", if is_valid { "성공" } else { "실패" });
// 최종 결과 계산 (예: 모든 당사자 입력의 합)
let mut total_value = Fr::from(0u64);
for (_, value) in expected_evaluations.iter() {
total_value += value;
}
println!("최종 MPC 결과: {:?}", total_value);
Ok(())
}
```

결론

이 문서에서는 arkworks-rs/poly-commit 라이브러리의 실제 사용법을 단계별로 설명했습니다. 기본적인 setup-commit-open-verify 워크플로우부터 고급 기능인 batch operations, hiding commitments, degree bounds까지 다양한 사용 사례를 다뤘습니다.

각 스킴의 특성을 이해하고 적절한 상황에서 올바른 스킴을 선택하는 것이 중요합니다. 성능이 최우선이라면 KZG10 기반 스킴을, transparency가 중요하다면 IPA나 linear codes 기반 스킴을 고려해보세요.

실제 운영 환경에서는 보안 파라미터 설정, 메모리 사용량 최적화, 에러 처리 등을 추가로 고려해야 합니다. 이 문서의 예제들을 기반으로 자신의 사용 사례에 맞게 적절히 수정하여 사용하시기 바랍니다.

관련 문서

이론적 배경

- **프로젝트 개요**: 라이브러리 전체 구조와 환경 설정 방법
- **핵심 인터페이스**: 예제에서 사용하는 트레이트와 데이터 구조의 상세 분석

구현 세부사항

- **KZG10 구현**: 예제에서 주로 사용하는 KZG10 스킴의 내부 동작
- **Marlin PC 구현**: Degree bound 예제에서 사용하는 Marlin PC의 구현 세부사항
- **기타 스킴들**: 다른 스킴들의 사용법과 선택 기준

심화 학습

- **암호학적 배경**: 예제에서 다루는 보안 고려사항과 매개변수 설정
- **용어집**: 예제에서 사용하는 모든 용어와 개념 정리